

Proyecto de Curso: Modelo de Siguiente Mejor Producto

1. Descripción general del proyecto

Este proyecto tiene como objetivo que los estudiantes desarrollen un flujo completo de datos y machine learning para construir un modelo de **Next Best Product** o **Mejor Siguiente Producto a Ofrecer**.

El propósito del modelo es predecir qué producto financiero tiene mayor probabilidad de ser adquirido por un cliente en el siguiente periodo, a partir de su información histórica, productos actuales, segmento, comportamiento mensual y características del producto candidato.

El proyecto debe simular un proceso real de analítica y ciencia de datos, desde la recepción de archivos crudos hasta la generación de predicciones listas para ser consumidas por un área comercial.

2. Objetivo del proyecto

Construir una solución de datos y machine learning que permita:

1. Recibir archivos CSV normalizados en una capa inicial de datos crudos.
 2. Convertir los archivos CSV a formato Parquet.
 3. Organizar los datos en una arquitectura por capas: **Raw**, **Bronze**, **Silver** y **Gold**.
 4. Construir un dataset final para entrenamiento del modelo.
 5. Entrenar un modelo supervisado de Next Best Product.
 6. Generar una predicción para cada cliente y producto candidato.
 7. Recomendar el producto con mayor probabilidad de adquisición.
-

3. Contexto del negocio

Una institución financiera desea mejorar sus campañas comerciales recomendando a cada cliente el producto más adecuado para ofrecerle.

En lugar de enviar campañas generales a todos los clientes, se busca utilizar datos históricos para responder preguntas como:

- ¿Qué productos tiene más probabilidad de adquirir este cliente?

- ¿Qué clientes tienen mayor oportunidad comercial?
- ¿Qué productos deberían priorizarse en una campaña?
- ¿Qué clientes ya tienen un producto y no deberían recibir una oferta repetida?

El modelo no solo debe predecir si un cliente adquirirá o no un producto, sino también permitir ordenar los productos candidatos de mayor a menor probabilidad.

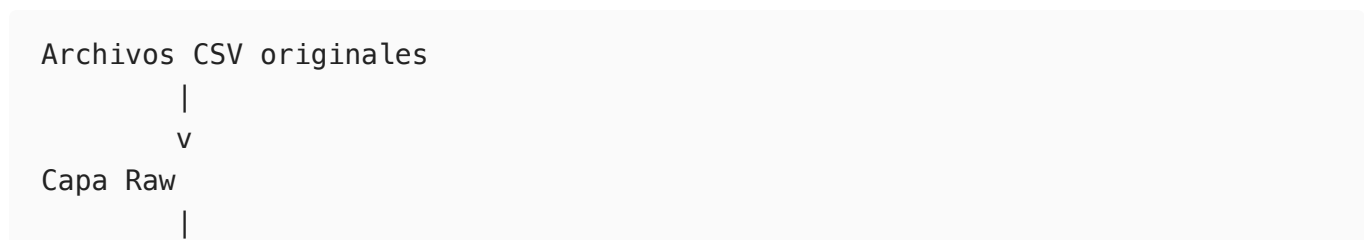
4. Archivos base del proyecto

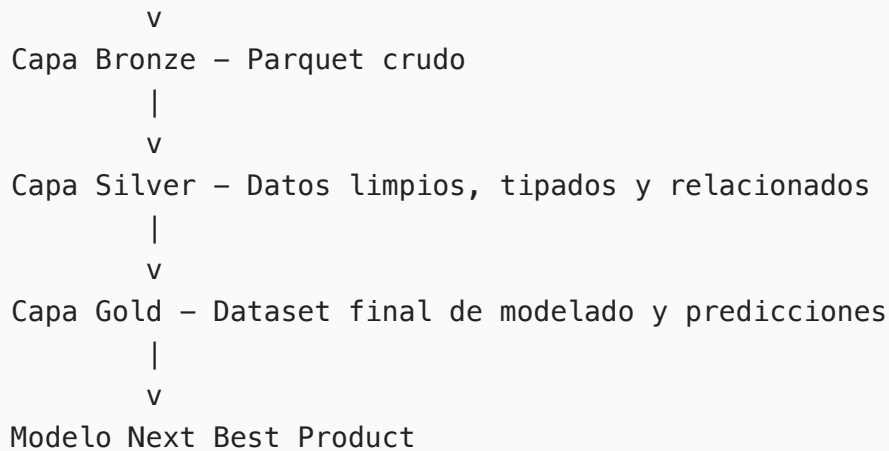
El proyecto parte de archivos CSV normalizados. Estos archivos representan entidades separadas y permiten construir el dataset final de modelado.

Archivo	Descripción
clientes.csv	Maestro de clientes. Contiene información estable del cliente.
provincias.csv	Catálogo de provincias.
segmentos.csv	Catálogo de segmentos comerciales.
productos.csv	Catálogo de productos financieros.
cliente_estado_mensual.csv	Estado mensual del cliente. Contiene variables que cambian por mes.
cliente_producto_mensual.csv	Relación cliente-producto-mes. Indica si un cliente tiene activo un producto.
cliente_producto_alta.csv	Eventos de adquisición de productos nuevos.
dataset_next_best_product.csv	Dataset final para entrenamiento o simulación del modelo.

5. Arquitectura de datos propuesta

El proyecto debe dividirse en capas de datos. Cada capa tiene un objetivo específico y representa un nivel distinto de transformación.





6. Capa Raw

6.1 Propósito

La capa **Raw** contiene los archivos originales tal como fueron recibidos. No se deben modificar los nombres de columnas, tipos de datos ni valores originales.

Esta capa sirve como respaldo y punto de partida del proceso.

6.2 Características

- Contiene archivos en formato CSV.
- No aplica transformaciones de negocio.
- No corrige datos faltantes.
- No cambia nombres de columnas.
- Conserva la estructura original.

6.3 Ejemplo de estructura

```
data/
  raw/
    clientes.csv
    provincias.csv
    segmentos.csv
    productos.csv
    cliente_estado_mensual.csv
    cliente_producto_mensual.csv
    cliente_producto_alta.csv
    dataset_next_best_product.csv
```

6.4 Actividades esperadas

Los estudiantes deben:

1. Crear la carpeta `data/raw`.
 2. Colocar allí todos los archivos CSV originales.
 3. Leer los archivos usando Python, pandas, PySpark o DuckDB.
 4. Validar que los archivos existan.
 5. Revisar número de filas, columnas y tipos inferidos.
 6. Identificar valores nulos y posibles inconsistencias.
-

7. Capa Bronze

7.1 Propósito

La capa **Bronze** contiene los mismos datos de Raw, pero convertidos a un formato más eficiente para procesamiento analítico: **Parquet**.

En esta capa todavía no se aplica una limpieza profunda. El objetivo principal es pasar de CSV a Parquet y conservar la trazabilidad de los datos originales.

7.2 Características

- Formato recomendado: Parquet.
- Los datos siguen siendo cercanos al origen.
- Puede incluir columnas técnicas como fecha de carga o nombre del archivo fuente.
- Mejora el rendimiento de lectura y escritura frente a CSV.

7.3 Ejemplo de estructura

```
data/  
  bronze/  
    clientes.parquet  
    provincias.parquet  
    segmentos.parquet  
    productos.parquet  
    cliente_estado_mensual.parquet  
    cliente_producto_mensual.parquet  
    cliente_producto_alta.parquet  
    dataset_next_best_product.parquet
```

7.4 Actividades esperadas

Los estudiantes deben:

1. Leer cada archivo CSV desde `data/raw`.
2. Convertir cada archivo a Parquet.
3. Guardar los archivos resultantes en `data/bronze`.
4. Agregar, si se desea, columnas de auditoría:
 - `fecha_carga`
 - `archivo_origen`
 - `capa_origen`
5. Validar que el número de registros sea igual entre Raw y Bronze.

7.5 Ejemplo conceptual

```
import pandas as pd

clientes = pd.read_csv("data/raw/clientes.csv")
clientes.to_parquet("data/bronze/clientes.parquet", index=False)
```

8. Capa Silver

8.1 Propósito

La capa **Silver** contiene datos limpios, tipados, relacionados y preparados para análisis.

Aquí los estudiantes deben aplicar reglas de calidad, convertir tipos de datos, resolver valores faltantes y preparar las relaciones entre tablas.

8.2 Características

- Datos limpios y estandarizados.
- Tipos de datos corregidos.
- Fechas convertidas correctamente.
- Variables categóricas normalizadas.
- Relaciones entre tablas verificadas.
- Llaves primarias y foráneas revisadas.

8.3 Ejemplo de estructura

```
data/  
  silver/  
    clientes.parquet  
    provincias.parquet  
    segmentos.parquet  
    productos.parquet  
    cliente_estado_mensual.parquet  
    cliente_producto_mensual.parquet  
    cliente_producto_alta.parquet
```

8.4 Transformaciones esperadas

Clientes

- Validar que `id_cliente` no sea nulo.
- Eliminar duplicados por cliente si existen.
- Convertir `fecha_alta` a tipo fecha.
- Crear una variable de antigüedad del cliente si aplica.

Estado mensual del cliente

- Convertir `fecha_corte` a fecha.
- Validar la relación con `clientes.csv`.
- Validar la relación con `provincias.csv` usando `cod_prov`.
- Validar la relación con `segmentos.csv` usando `id_segmento`.
- Imputar o marcar valores faltantes de renta, edad o segmento.

Productos

- Validar que cada `id_producto` sea único.
- Revisar que cada producto tenga nombre y categoría.
- Estandarizar nombres de productos y categorías.

Cliente-producto mensual

- Validar que `estado_producto` sea 0 o 1.
- Validar la relación con `clientes.csv`.
- Validar la relación con `productos.csv`.
- Revisar duplicados por `id_cliente`, `fecha_corte`, `origen_datos` e `id_producto`.

Altas de producto

- Validar que cada alta represente una adquisición real.
 - Confirmar que el producto no estaba activo en el mes anterior.
 - Confirmar que el producto aparece activo en el mes actual.
-

9. Capa Gold

9.1 Propósito

La capa **Gold** contiene los datos listos para el consumo del modelo y del negocio.

En esta capa se construye el dataset final para entrenamiento, validación, predicción y recomendación.

9.2 Estructura sugerida

```
data/  
  gold/  
    dataset_entrenamiento_nbp.parquet  
    dataset_prediccion_nbp.parquet  
    predicciones_next_best_product.parquet  
    recomendaciones_next_best_product.parquet
```

9.3 Dataset de entrenamiento

El dataset de entrenamiento debe tener una fila por:

```
cliente + fecha_corte + producto_candidato
```

Cada fila representa una pregunta de negocio:

Para este cliente, en este mes, ¿adquirirá este producto candidato en el siguiente periodo?

9.4 Variable objetivo

La variable objetivo se llama `y`.

```
y = 1 si el cliente adquiere el producto candidato en el siguiente periodo.  
y = 0 si el cliente no adquiere el producto candidato en el siguiente  
periodo.
```

La lógica general es:

Producto no estaba activo en el mes t
Producto aparece activo en el mes $t + 1$
Entonces $y = 1$

9.5 Variables sugeridas

Grupo	Variables de ejemplo
Cliente	edad, antigüedad, renta, segmento, provincia
Relación bancaria	cantidad_productos_actuales, ind_actividad_cliente
Productos actuales	tiene_cuenta, tiene_credito, tiene_tarjeta, tiene_inversion
Producto candidato	id_producto_candidato, categoría_producto_candidato
Historial	productos activos en meses anteriores, altas previas
Target	y

9.6 Dataset de predicción

El dataset de predicción debe tener la misma estructura de variables que el dataset de entrenamiento, pero sin la variable y o con y nula.

Debe contener los productos candidatos que se desean evaluar para cada cliente.

10. Desarrollo del modelo

10.1 Tipo de problema

El problema puede abordarse como una clasificación supervisada binaria.

Cada fila del dataset representa un producto candidato para un cliente.

El modelo aprende a estimar:

```
P(adquirir producto candidato | cliente, mes, historial, producto)
```

10.2 Modelos recomendados

Los estudiantes pueden iniciar con modelos simples y luego comparar con modelos más avanzados.

Nivel	Modelo	Comentario
Básico	Regresión logística	Fácil de explicar e interpretar.
Intermedio	Árbol de decisión	Útil para entender reglas de decisión.
Intermedio	Random Forest	Mejor capacidad predictiva que un árbol simple.
Avanzado	XGBoost, LightGBM o CatBoost	Recomendado para datos tabulares.

10.3 División de datos

Se recomienda realizar una separación temporal.

Ejemplo:

```
Meses históricos antiguos -> entrenamiento
Meses recientes -> validación
Último mes disponible -> prueba o simulación
```

No se recomienda una separación aleatoria si el objetivo es simular una predicción futura.

10.4 Métricas sugeridas

Métrica	Uso
AUC-ROC	Evalúa capacidad general de separación.
Average Precision / AUC-PR	Útil si hay pocos productos adquiridos frente a muchos no adquiridos.
Precision@K	Mide calidad de las recomendaciones top K.
Recall@K	Mide cuántas adquisiciones reales aparecen dentro de las recomendaciones.
Lift	Compara el modelo contra una selección aleatoria.

11. Generación de predicciones

11.1 Proceso de scoring

Para generar predicciones, los estudiantes deben:

1. Seleccionar los clientes a evaluar.
2. Generar todos los productos candidatos posibles.
3. Excluir productos que el cliente ya tiene activos, si la regla de negocio lo requiere.
4. Construir las mismas variables usadas en entrenamiento.
5. Aplicar el modelo entrenado.
6. Obtener una probabilidad para cada cliente y producto candidato.

11.2 Salida esperada de predicción

Archivo sugerido:

```
data/gold/predicciones_next_best_product.parquet
```

Columnas sugeridas:

Campo	Descripción
id_cliente	Cliente evaluado.
fecha_corte	Fecha de observación.
id_producto_candidato	Producto evaluado.
nombre_producto	Nombre del producto.
categoria_producto	Categoría del producto.
probabilidad_adquisicion	Probabilidad estimada por el modelo.
ranking_producto	Posición del producto para el cliente.

11.3 Recomendación final

Archivo sugerido:

```
data/gold/recomendaciones_next_best_product.parquet
```

Debe contener el producto recomendado para cada cliente.

Columnas sugeridas:

Campo	Descripción
id_cliente	Cliente evaluado.
fecha_corte	Fecha de recomendación.

Campo	Descripción
producto_recomendado	Producto con mayor probabilidad.
categoria_producto	Categoría del producto recomendado.
probabilidad	Probabilidad estimada.
posicion_ranking	Ranking del producto.

12. Flujo completo del proyecto

1. Ingesta de CSV
raw/*.csv
2. Conversión a Parquet
raw/*.csv -> bronze/*.parquet
3. Limpieza y validación
bronze/*.parquet -> silver/*.parquet
4. Construcción de features
silver/*.parquet -> gold/dataset_entrenamiento_nbp.parquet
5. Entrenamiento del modelo
dataset_entrenamiento_nbp.parquet -> modelo entrenado
6. Scoring o predicción
dataset_prediccion_nbp.parquet -> predicciones_next_best_product.parquet
7. Recomendación final
predicciones -> recomendaciones_next_best_product.parquet

13. Entregables esperados del estudiante

Cada estudiante o grupo debe entregar:

1. Estructura de carpetas del proyecto.
2. Scripts o notebooks de procesamiento.
3. Archivos Parquet generados en Bronze, Silver y Gold.
4. Análisis exploratorio básico.

5. Dataset final de entrenamiento.
 6. Modelo entrenado.
 7. Métricas de evaluación.
 8. Archivo de predicciones.
 9. Archivo de recomendaciones finales.
 10. Breve documento explicando decisiones técnicas y de negocio.
-

14. Estructura sugerida del repositorio

```
next-best-product-project/  
  data/  
    raw/  
    bronze/  
    silver/  
    gold/  
  
  notebooks/  
    01_ingesta_raw.ipynb  
    02_convertir_bronze.ipynb  
    03_limpieza_silver.ipynb  
    04_construccion_gold.ipynb  
    05_entrenamiento_modelo.ipynb  
    06_prediccion_recomendacion.ipynb  
  
  src/  
    data_processing.py  
    feature_engineering.py  
    train_model.py  
    predict.py  
  
  models/  
    modelo_next_best_product.pkl  
  
  docs/  
    diccionario_datos.md  
    informe_modelo.md  
  
  README.md
```

15. Reglas importantes del proyecto

15.1 Evitar fuga de información

Los estudiantes no deben usar información futura para construir variables del periodo actual.

Incorrecto:

```
Usar productos adquiridos en t+1 como variable para predecir t+1.
```

Correcto:

```
Usar información disponible hasta el mes t para predecir adquisición en t+1.
```

15.2 No usar la variable objetivo como feature

La columna `y` solo se utiliza para entrenar y evaluar el modelo. No debe usarse como variable predictora.

15.3 Excluir productos ya contratados

En la etapa de recomendación, se debe revisar si tiene sentido ofrecer productos que el cliente ya tiene.

Regla sugerida:

```
Si el cliente ya tiene el producto activo, no se recomienda ese producto.
```

15.4 Mantener consistencia entre entrenamiento y predicción

Las variables usadas para predecir deben ser las mismas que se usaron para entrenar el modelo.

16. Criterios de evaluación del curso

Criterio	Peso sugerido
Organización del proyecto por capas	15%
Conversión correcta a Parquet	10%
Limpieza y validación de datos	15%

Criterio	Peso sugerido
Construcción correcta del dataset de entrenamiento	20%
Entrenamiento y evaluación del modelo	20%
Generación de predicciones y recomendaciones	15%
Documentación y claridad del proyecto	5%

17. Resultado esperado

Al finalizar el proyecto, los estudiantes deberán tener una solución capaz de generar una tabla como la siguiente:

id_cliente	producto_recomendado	probabilidad	ranking
10001	Tarjeta de crédito	0.72	1
10002	Cuenta de ahorro	0.65	1
10003	Fondo de inversión	0.58	1

Esta tabla representa la recomendación final del modelo para cada cliente.

18. Conclusión

Este proyecto permite que los estudiantes practiquen un flujo completo de ciencia de datos aplicado a un caso financiero realista.

El valor principal del ejercicio no está solo en entrenar un modelo, sino en comprender cómo los datos deben organizarse, limpiarse, transformarse y gobernarse antes de llegar a una predicción útil para el negocio.

La arquitectura por capas permite separar responsabilidades, mejorar la trazabilidad y construir una solución más cercana a un entorno profesional de datos.